

T5 Exercises – Fateme Firouznia

1. Please explain the use cases of soft and hard links. When should we use a soft link, and when is a hard link more appropriate?

A **hard link** is an additional directory entry that points to the **same inode** as the original file, so it creates another name for the same underlying data rather than copying the file. Hard links are most appropriate when you want multiple filenames for the same content **within the same filesystem**, and you want the data to remain accessible as long as at least one name still exists. Deleting one filename only decrements the inode's link count; the file's data is removed only when the link count reaches zero. Hard links generally cannot be created for directories (to avoid cycles in the directory structure) and cannot cross filesystem boundaries.

A **soft link (symbolic link)** is a separate filesystem object with its **own inode** whose content is a **path** to the target. Soft links are appropriate when you need to link to **directories**, create convenient aliases for long paths, or create links that can **cross filesystems/mount points**. The main limitation is that a symlink depends on the target path: if the target is moved or removed, the symlink becomes **dangling (broken)** and path resolution fails.

```
Last login: Fri Dec 19 13:24:16 2025 from 192.168.92.1
mohammad@template:~$ mkdir -p ~/link-lab && cd ~/link-lab
mohammad@template:~/link-lab$ echo "hello" > file.txt
mohammad@template:~/link-lab$ ln file.txt hardlink.txt
mohammad@template:~/link-lab$ ln -s file.txt softlink.txt
mohammad@template:~/link-lab$ ls -li file.txt hardlink.txt softlink.txt
393418 -rw-rw-r-- 2 mohammad mohammad 6 Dec 20 11:37 file.txt
393418 -rw-rw-r-- 2 mohammad mohammad 6 Dec 20 11:37 hardlink.txt
393419 lrwxrwxrwx 1 mohammad mohammad 8 Dec 20 11:37 softlink.txt -> file.txt
mohammad@template:~/link-lab$ stat file.txt
  File: file.txt
  Size: 6                Blocks: 8          IO Block: 4096   regular file
Device: 252,0    Inode: 393418        Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/mohammad)   Gid: ( 1000/mohammad)
Access: 2025-12-20 11:37:03.858441804 +0000
Modify: 2025-12-20 11:37:03.858441804 +0000
Change: 2025-12-20 11:37:10.488408550 +0000
 Birth: 2025-12-20 11:37:03.858441804 +0000
mohammad@template:~/link-lab$ readlink softlink.txt
file.txt
mohammad@template:~/link-lab$ rm file.txt
mohammad@template:~/link-lab$ ls -l hardlink.txt softlink.txt
-rw-rw-r-- 1 mohammad mohammad 6 Dec 20 11:37 hardlink.txt
lrwxrwxrwx 1 mohammad mohammad 8 Dec 20 11:37 softlink.txt -> file.txt
mohammad@template:~/link-lab$ cat hardlink.txt
hello
mohammad@template:~/link-lab$ cat softlink.txt
cat: softlink.txt: No such file or directory
mohammad@template:~/link-lab$
```

In my test, file.txt and hardlink.txt share the same inode (**393418**), while softlink.txt has a different inode (**393419**) and points to the target path file.txt. After removing file.txt, hardlink.txt still prints the content (hello), while softlink.txt fails because its target no longer exists.

2. Describe the inode relationship of soft links. How do inodes work with soft and hard links? Please show inode numbers and explain how the inode accesses the data on the hard disk.

An inode is a filesystem data structure that stores a file's metadata and the information required to locate its data on disk. File names are not stored in the inode; instead, directory entries map names to inode numbers.

An inode typically contains file type, permissions, ownership, timestamps, file size, and pointers (or extents) that describe where the file's data blocks are located on disk.

When a file is accessed, the operating system resolves the filename to an inode, then uses the inode's block mapping information to read the actual data.

For a regular file, the inode points to the data blocks that contain the file's content. For a symbolic link, the inode does not point to the target's data; instead, its data contains a path string that refers to the target. Tools such as `ls -i` and `stat` can be used to inspect inode numbers and metadata, while `readlink` shows the path stored in a symbolic link. On my system, `stat file.txt` shows inode 393418, which is used by both file.txt and its hard link.

3. What is the /opt directory used for? How does it differ from /etc and /sbin in terms of purpose and usage?

The /opt directory is intended for **optional or third-party software** that is not part of the core operating system packages. It is commonly used for vendor-supplied or self-contained applications that should remain isolated from the standard system directories.

The /etc directory contains **system-wide configuration files**, typically plain text, that define how the operating system and its services are configured. It does not store binaries or application data.

The /sbin directory contains **system administration binaries**, which are tools used for managing and maintaining the system, such as filesystem, networking, and boot-related utilities. On modern systems, /sbin is often merged with or linked to /usr/sbin, but its administrative role remains the same.

In summary, /opt is about where optional software is installed, /etc defines how the system is configured, and /sbin provides the tools used to administer the system.

4. Compare the locate and find commands. Which one is faster, and why?

The find command searches the filesystem **in real time** by traversing directories and inspecting inode information directly from disk. Because it scans the filesystem at query time, it always returns up-to-date results, but it can be slow on large directory trees due to disk I/O.

The locate command searches a **prebuilt database** of file paths that is generated by the updatedb utility. Since it does not scan the filesystem when a query is executed, it is usually much faster than find. However, its results may be outdated if the database has not been recently updated.

In practice, find is preferred when accuracy and real-time results are required, while locate is useful for quick searches when slightly stale results are acceptable.

Unlike find, locate does not inspect filesystem metadata at query time; it relies entirely on its database.

5. Please explain how your device's updatedb.conf is configured. How does it affect the behavior of the locate command?

The updatedb.conf file controls how the updatedb utility builds the database that is used by the locate command. It defines which filesystems, paths, and filenames should be excluded from indexing.

Settings such as PRUNEFS specify filesystem types that should not be indexed, for example virtual or temporary filesystems like proc, sysfs, or tmpfs. PRUNEPATHS defines directory paths that are excluded from the database, such as temporary directories or removable media.

These exclusions improve performance and avoid indexing volatile or irrelevant locations. As a result, locate will never return files from pruned filesystems or paths, even if they exist on disk. This behavior is intentional and reflects the configuration defined in updatedb.conf.

On my system, updatedb.conf excludes virtual filesystems to prevent unnecessary indexing overhead.

6. What is the difference between .bashrc and .bash_profile? What is the purpose of /etc/bash_completion?

The `.bashrc` file is executed for **interactive non-login shells**, such as when opening a new terminal window or starting a subshell. It is commonly used to define aliases, shell functions, prompt settings, and other interactive behavior.

The `.bash_profile` file is executed for **login shells**, for example when logging in via SSH or through a system console. It is typically used to set environment variables and session-wide configuration. In many setups, `.bash_profile` explicitly sources `.bashrc` so that interactive settings are also applied.

The `/etc/bash_completion` file provides **system-wide programmable tab completion** for many commands. When enabled, it enhances usability by allowing Bash to complete command names, options, and arguments automatically.

7. What kind of information is stored in `.bash_logout` and `.bash_login`? What are their typical use cases?

The `.bash_login` file is one of the startup files used by **login shells**. When a login shell starts, Bash checks for `.bash_profile`, then `.bash_login`, and then `.profile`, executing the first file it finds. The purpose of `.bash_login` is to initialize the login session by setting environment variables or starting session-related services.

The `.bash_logout` file is executed when a **login shell exits**. It is typically used for cleanup tasks such as removing temporary files, resetting the terminal, or performing other shutdown-related actions.

Unlike `.bashrc`, which can be executed multiple times during a session for interactive shells, `.bash_login` and `.bash_logout` are tied specifically to the start and end of a login session.

8. Please demonstrate how to customize the command prompt (PS1) in the `.bashrc` file.

The `PS1` variable defines the primary command prompt displayed by Bash. It controls what information is shown before each command, such as the username, hostname, and current working directory.

Since the prompt is only relevant for interactive shells, `PS1` is typically configured in the `.bashrc` file.

For example, adding the following line to ~/.bashrc customizes the prompt to display the username, hostname, and current directory:

```
PS1='\[CUSTOM]\u@\h:\w\$ '
```

```
Last login: Sat Dec 20 11:36:28 2025 from 192.168.92.1
mohammad@template:~$
```

```
GNU nano 7.2 /home/mohammad/.bashrc *
# If this is an xterm set the title to user@host:dir
case "$TERM" in
xterm*|rxvt*)
    PS1='\[CUSTO]\u@\h:\w\$'PS1"
    ;;
*)
    ;;
esac

# enable color support of ls and also add handy aliases
if [ -x /usr/bin/dircolors ]; then
    test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval "$(dircolors -b)"
    alias ls='ls --color=auto'
    #alias dir='dir --color=auto'
    #alias vdir='vdir --color=auto'

    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

# colored GCC warnings and errors
#export GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=01:quote=01'

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'

# Add an "alert" alias for long running commands. Use like so:
# sleep 10; alert
alias alert='notify-send --urgency=low -i "${[ $? = 0 ]} && echo terminal || echo error" "${history|tail -n1|sed -e '\''s/\^[0-9]\+.*$//;s/[]&|

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi
PS1='\[CUSTOM]\u@\h:\w\$ '
```

```
mohammad@template:~$ nano ~/.bashrc
mohammad@template:~$ source ~/.bashrc
[CUSTOM]mohammad@template:~$
```

After modifying the PS1 variable in .bashrc and reloading the file, the command prompt was updated as shown.

9. Write a Bash script that takes a webpage URL as input and downloads all the URLs (links) found on that page.

```
#!/bin/bash

# Check that a URL is provided
if [ -z "$1" ]; then
    echo "Usage: $0 <URL>"
    exit 1
fi

URL="$1"

# Download the HTML content of the webpage
HTML=$(curl -s "$URL")

# Extract absolute URLs from the page
LINKS=$(echo "$HTML" | grep -o 'https\?://[^\s]*')

# Download each extracted URL
for link in $LINKS; do
    wget "$link"
done
```

The script takes a webpage URL as a command-line argument. It uses curl to retrieve the HTML content of the page, then extracts absolute URLs (http and https) using grep. Each extracted URL is processed in a loop and downloaded using wget.

This approach is sufficient for a basic Bash exercise. While HTML parsing with regular expressions is not suitable for complex or production use cases, it is acceptable here to demonstrate command-line scripting, input handling, and control flow in Bash.

10. Please explain the use of the exit command and other common exit-related statements in Bash scripting.

The `exit` command is used to terminate a Bash script and return an exit status to the calling shell or process. An exit status of 0 indicates successful execution, while a non-zero value indicates an error or abnormal condition.

If `exit` is used without an argument, the script returns the exit status of the last executed command. The special variable `$?` can be used to inspect the exit code of the most recently executed command.

The `return` statement is used inside functions and exits only from the function, not from the entire script.

The `set -e` option causes a script to terminate immediately if any command returns a non-zero exit status, helping prevent execution after failures.

The `trap` statement allows a script to execute specified commands when it exits or when it receives certain signals, and is commonly used for cleanup operations.